

Generative Modeling

Wasserstein generative adversarial networks and friends

Tigran Ramazyan, Anwar Ibrahim, Fedor Pakhurov

Laboratory for methods of big data analysis



LAMBDA • HSE

Fall 2025

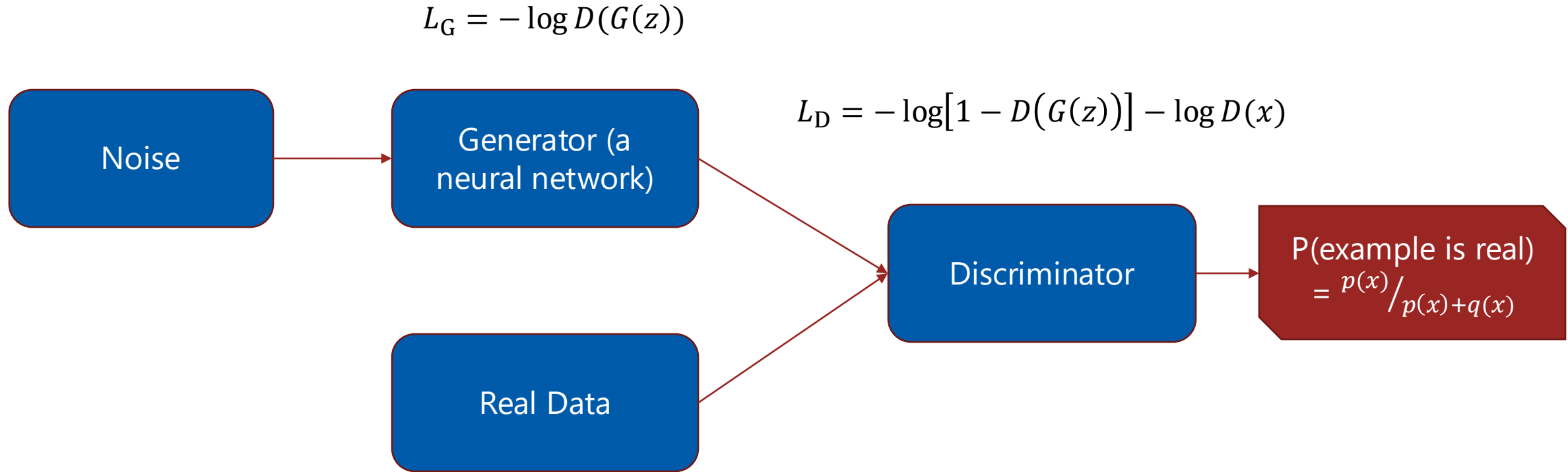
In this Lecture

- ▶ f-GANs
- ▶ Wasserstein Distance
 - Kantorovich-Rubenstein Duality.
- ▶ WGAN
 - Algorithm.
 - Gradient Penalty.
 - Biased Gradients.

Reminder



JS GAN scheme



JS GAN Summary

- ▶ Pros:
 - Can utilize power of back-prop.
 - No explicit intractable integral.
 - No MCMC needed.
- ▶ Cons:
 - Unclear stopping criteria
 - No explicit representation of PDF
 - Hard to train
 - No evaluation metric so hard to compare with other models
 - Easy to get trapped in local optima that memorize training data
 - Hard to invert generative model to get back latent z from generated x

f -GANs



Reminder: Variational Lower Bound

- ▶ For convex $f(\cdot)$, P and Q some distributions, we define f -divergence:

$$D_f(P||Q) = \int q(x) f\left(\frac{p(x)}{q(x)}\right) dx.$$

- ▶ This is bounded:

$$D_f(P||Q) \geq \max_{T(x)} \mathbb{E}_{x \sim P} T(x) - \mathbb{E}_{x \sim Q} f^*(T(x)),$$

Name	$D_f(P Q)$	Generator $f(u)$	$T^*(x)$
Kullback-Leibler	$\int p(x) \log \frac{p(x)}{q(x)} dx$	$u \log u$	$1 + \log \frac{p(x)}{q(x)}$
Reverse KL	$\int q(x) \log \frac{q(x)}{p(x)} dx$	$-\log u$	$-\frac{q(x)}{p(x)}$
Pearson χ^2	$\int \frac{(q(x)-p(x))^2}{p(x)} dx$	$(u-1)^2$	$2\left(\frac{p(x)}{q(x)} - 1\right)$
Squared Hellinger	$\int \left(\sqrt{p(x)} - \sqrt{q(x)}\right)^2 dx$	$(\sqrt{u} - 1)^2$	$\left(\sqrt{\frac{p(x)}{q(x)}} - 1\right) \cdot \sqrt{\frac{q(x)}{p(x)}}$
Jensen-Shannon	$\frac{1}{2} \int p(x) \log \frac{2p(x)}{p(x)+q(x)} + q(x) \log \frac{2q(x)}{p(x)+q(x)} dx$	$-(u+1) \log \frac{1+u}{2} + u \log u$	$\log \frac{2p(x)}{p(x)+q(x)}$

Variational Divergence Minimization

$$D_f(P||Q) \geq \max_{T(x)} \mathbb{E}_{x \sim P} T(x) - \mathbb{E}_{x \sim Q} f^*(T(x)),$$

- ▶ Work in GAN paradigm:
 - generator $x \sim Q: x = G_\theta(z)$;
 - test function $T(x)$.

$$\min_{\theta} \max_{\omega} F(\omega, \theta) = \mathbb{E}_{x \sim P} T_{\omega}(x) - \mathbb{E}_{x \sim G_{\theta}(z)} f^*(T_{\omega}(x))$$

- ▶ To have wider range of functions:

$$T_{\omega}(x) = g_f(V_{\omega}(x)),$$

here $g_f: \mathbb{R} \rightarrow \text{dom}_{f^*}$ is output activation function for f -divergence used

S. Nowozin et al. f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization

Output activation function

$$F(\omega, \theta) = \mathbb{E}_{x \sim P} g_f(V_\omega(x)) - \mathbb{E}_{x \sim G_\theta(z)} f^*(g_f(V_\omega(x)))$$

choice of output activation function is somewhat arbitrary.

Name	Output activation g_f
Kullback-Leibler (KL)	v
Reverse KL	$-\exp(-v)$
Pearson χ^2	v
Squared Hellinger	$1 - \exp(-v)$
Jensen-Shannon	$\log(2) - \log(1 + \exp(-v))$
GAN	$-\log(1 + \exp(-v))$

Example: GAN objective

$$F(\omega, \theta) = \mathbb{E}_{x \sim P} g_f(V_\omega(x)) - \mathbb{E}_{x \sim G_\theta(z)} f^*(g_f(V_\omega(x)))$$



$$\begin{aligned} g_{GAN} &= -\log(1 + \exp(-v)) \\ f^*(t) &= -\log(1 - \exp(t)) \end{aligned}$$

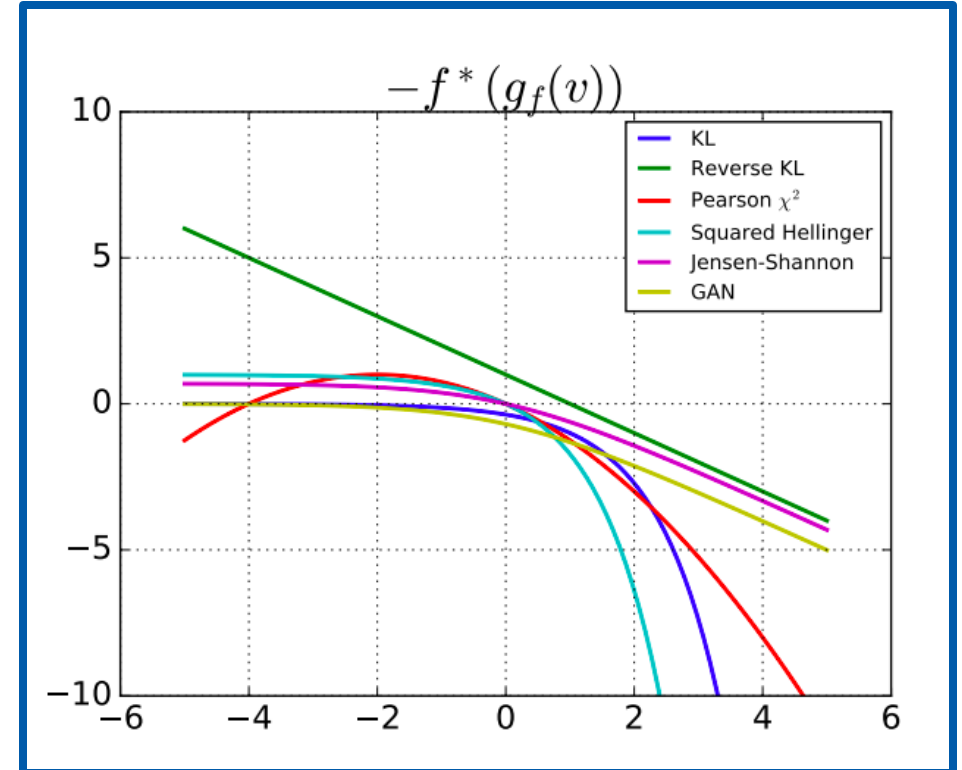
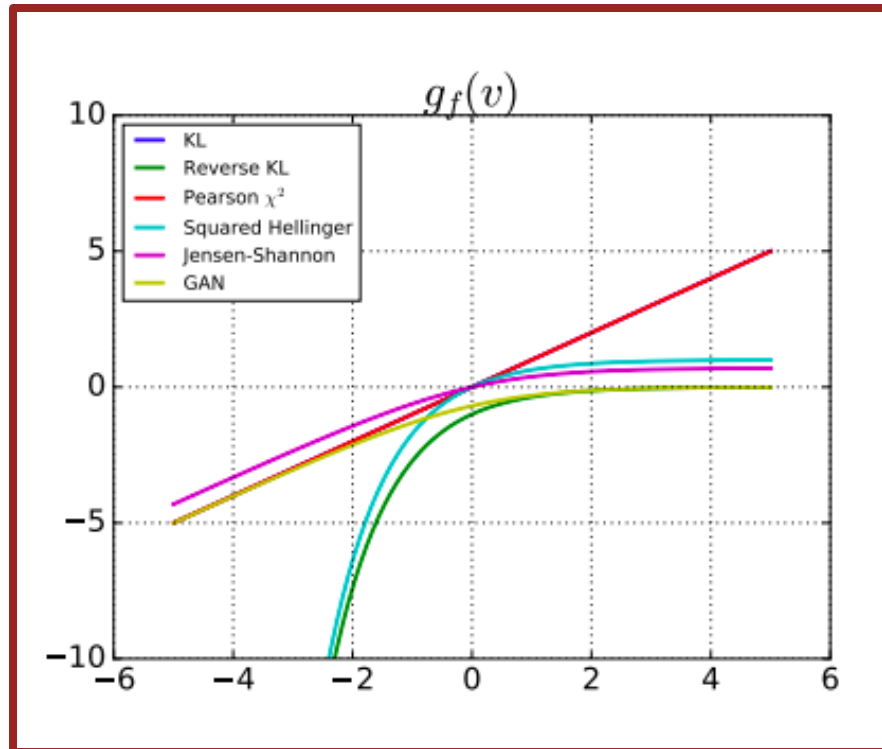
$$F(\omega, \theta) = \mathbb{E}_{x \sim P} \log D_\omega(x) - \mathbb{E}_{x \sim G_\theta(z)} \log(1 - D_\omega(x)),$$

for the last nonlinearity in the discriminator taken as the sigmoid

$$D_\omega(x) = 1/(1 + e^{-V_\omega(x)})$$

Variational Divergence Minimization

$$\min_{\theta} \max_{\omega} F(\omega, \theta) = \mathbb{E}_{x \sim P} \underline{g_f(V_{\omega}(x))} - \mathbb{E}_{x \sim G_{\theta}(z)} \underline{f^*(g_f(V_{\omega}(x)))}$$



f -GAN results

Training divergence	KDE $\langle LL \rangle$ (nats)	$\pm$ SEM
Kullback-Leibler	416	5.62
Reverse Kullback-Leibler	319	8.36
Pearson χ^2	429	5.53
Neyman χ^2	300	8.33
Squared Hellinger	-708	18.1
Jeffrey	-2101	29.9
Jensen-Shannon	367	8.19
GAN	305	8.97
Variational Autoencoder [18]	445	5.36
KDE MNIST train (60k)	502	5.99

Table 4: Kernel Density Estimation evaluation on the MNIST test data set. Each KDE model is build from 16,384 samples from the learned generative model. We report the mean log-likelihood on the MNIST test set ($n = 10,000$) and the standard error of the mean. The KDE MNIST result is using 60,000 MNIST training images to fit a single KDE model.



Figure 2: MNIST model samples trained using KL, reverse KL, Hellinger, Jensen from top to bottom.

f -GAN Discussion

- ▶ Using f -GAN approach, one can estimate any f -divergence.
- ▶ Construction has some freedom in choice of function.
- ▶ Using different f -divergence leads to very different learning dynamics.
- ▶ Does not solve mode collapse problem.
- ▶ We need a better way to train GANs.

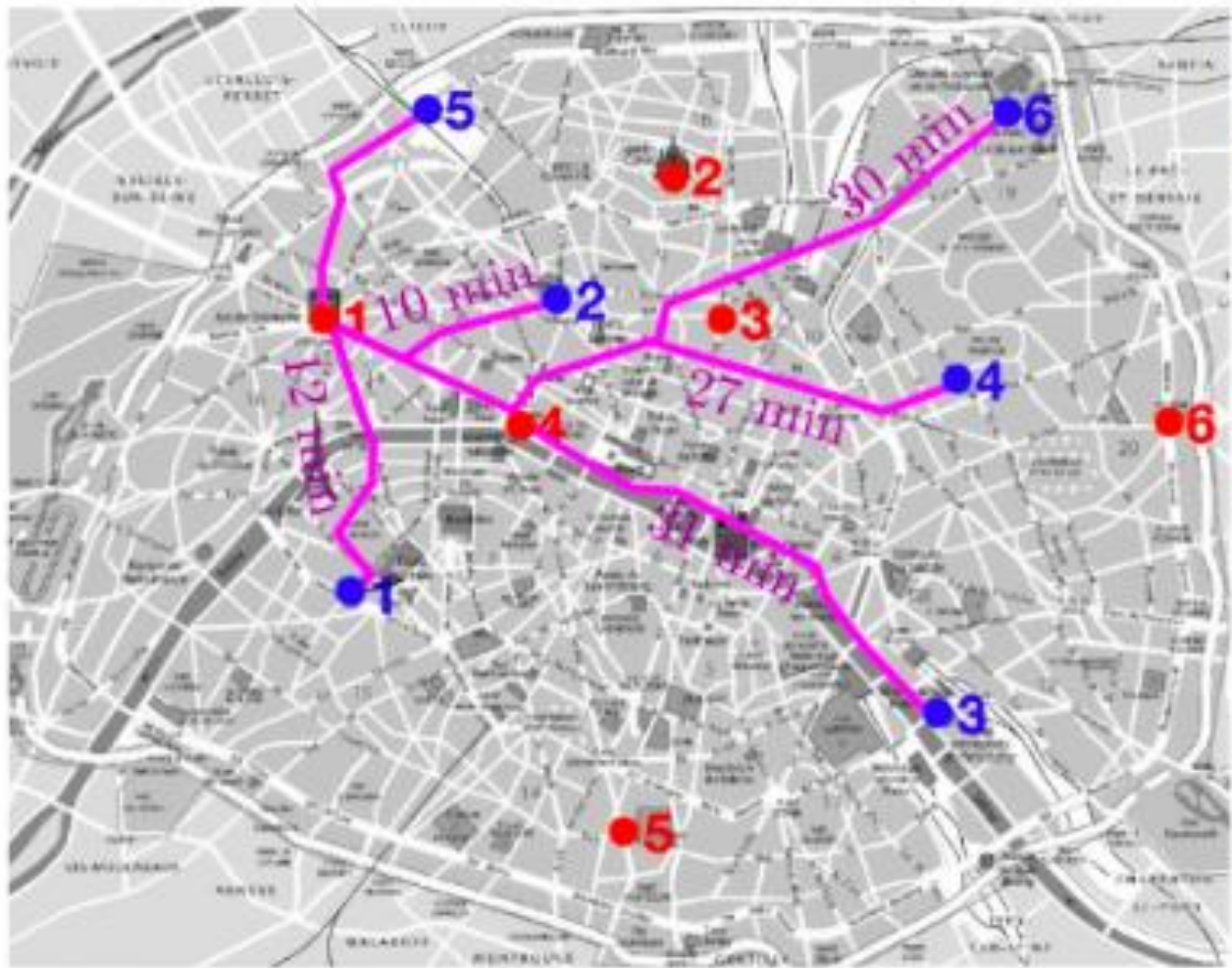
Parisian Bakeries



Problem Statement



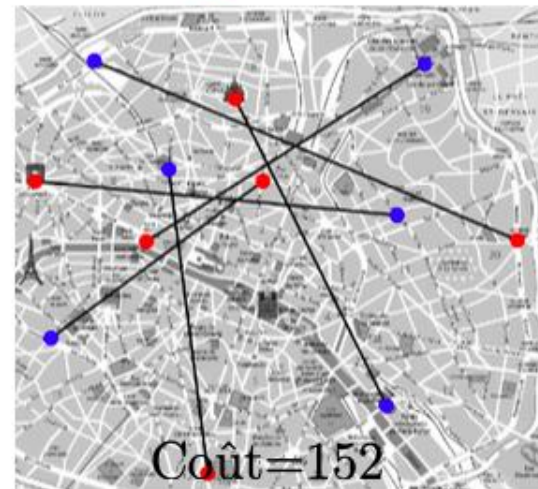
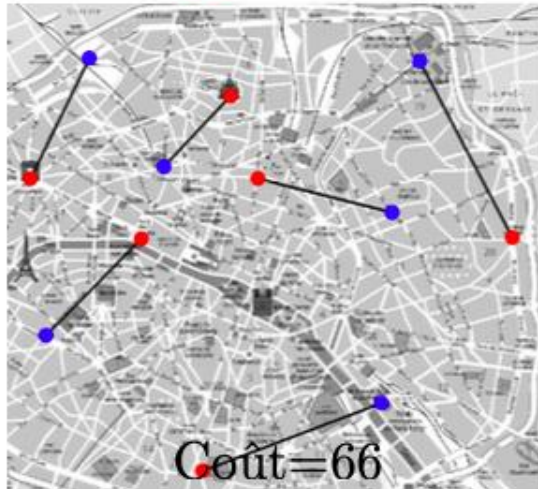
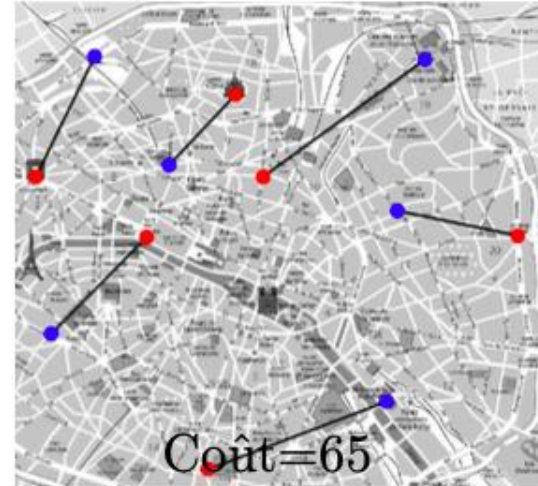
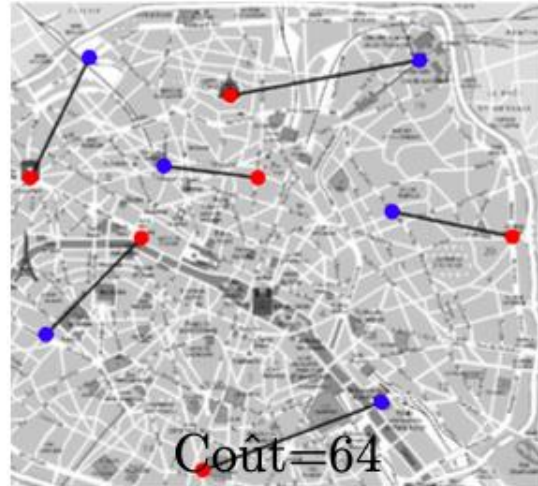
Delivery Optimization



c_j	y_1	y_2	y_3	y_4	y_5	y_6
x_1	12	10	31	27	10	30
x_2	22	7	25	15	11	14
x_3	19	7	19	10	15	15
x_4	10	6	21	19	14	24
x_5	15	23	14	24	31	34
x_6	35	26	16	9	34	15

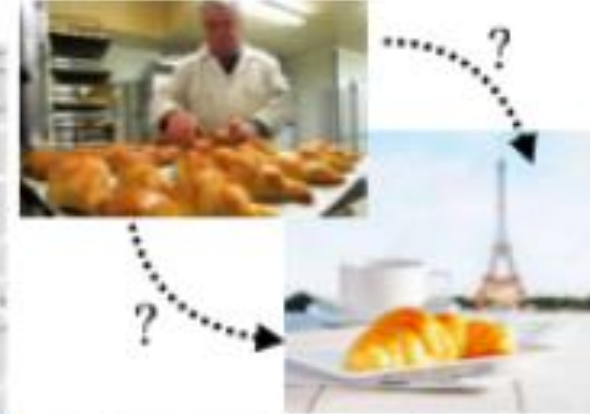
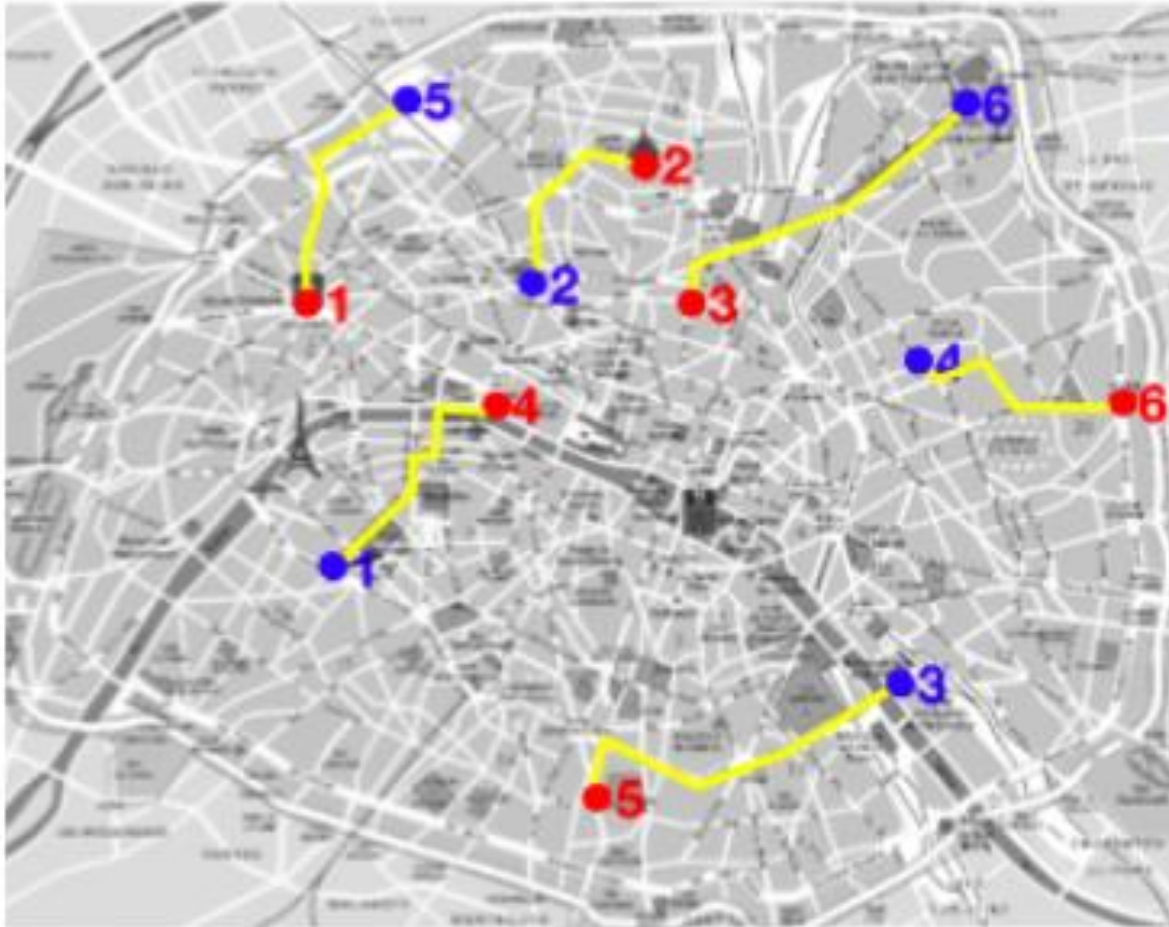
given a set of N bakeries and M cafes, what is the optimal way to transport loaves of bread between them?

Delivery Solutions



We can estimate different possibilities, using the same matrix of costs.

Optimal Delivery



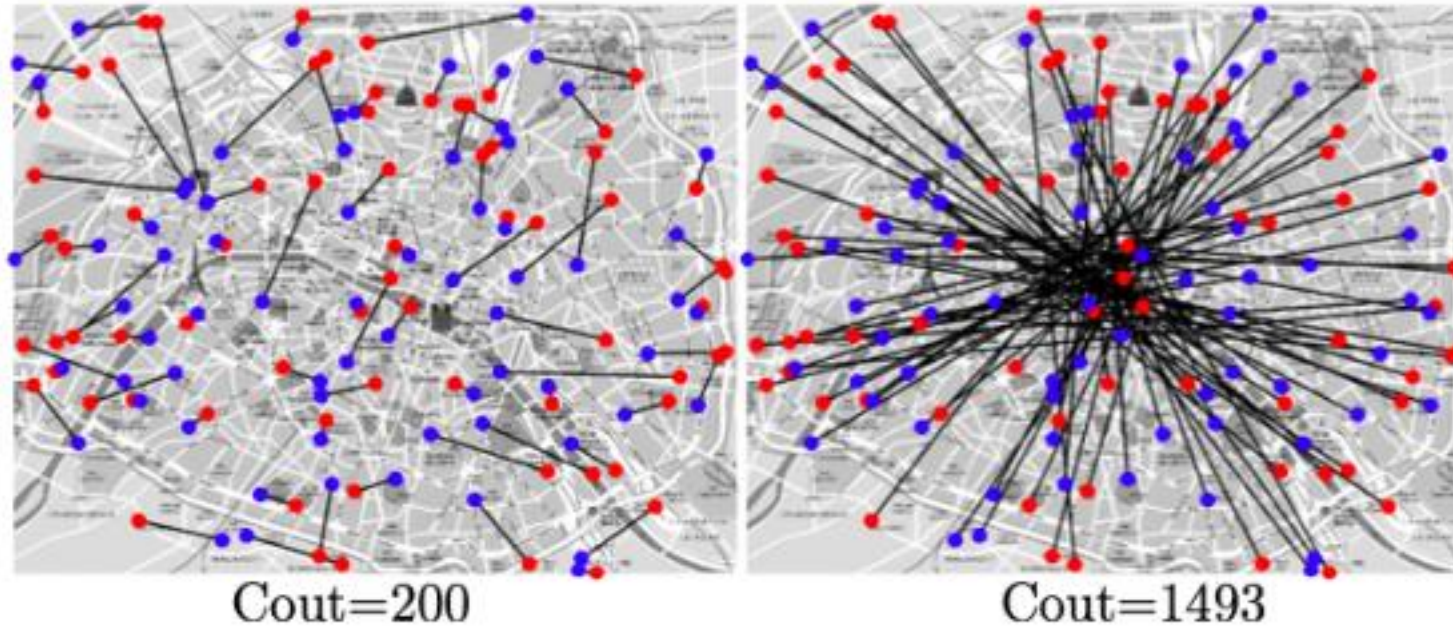
c_j	y_1	y_2	y_3	y_4	y_5	y_6
x_1	12	10	31	27	10	30
x_2	22	7	25	15	11	14
x_3	19	7	19	10	15	15
x_4	10	6	21	19	14	24
x_5	15	23	14	24	31	34
x_6	35	26	16	9	34	15

$$\text{Price} = 10 + 7 + 15 + 10 + 14 + 9 = 65 \text{ min}$$

Problem Statement: Monge

We thus need to solve a problem:

$$\min_{\sigma \in \text{Perm}_n} \sum_{i=1}^n C_{i, \sigma(i)}$$



The number of calculations rises as factorial.

Kantorovich: Add Bread Masses

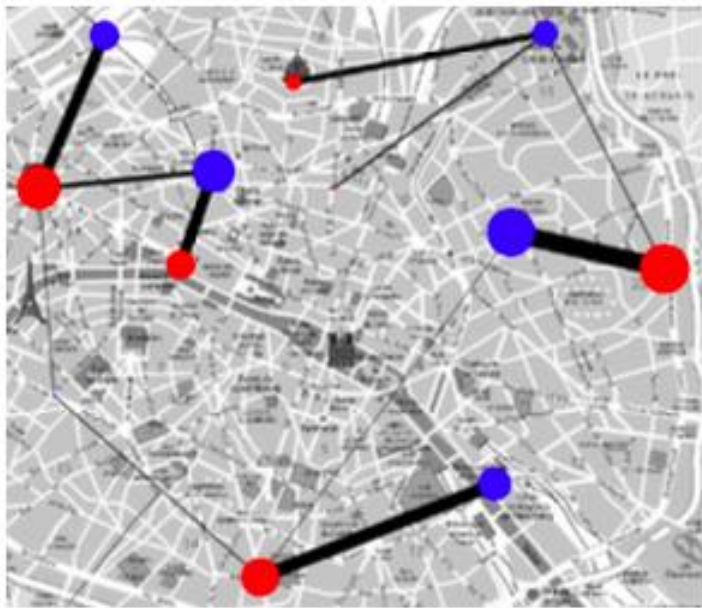
- ▶ $p_i, i \in 1 \dots N$ the mass of bread held by each bakery;
- ▶ $q_j, j \in 1 \dots M$ the mass of bread desired by each cafe;
- ▶ $\sum p_i = \sum q_j$, bread produced = bread needed
- ▶ x_i, y_j the positions of bakeries and cafes;
- ▶ $P_{i,j}$ transport of mass from i to j .
- ▶ $p_i = \sum_j P_{i,j}; q_j = \sum_i P_{i,j}$ limiting capacity exists.

Paris deliveries

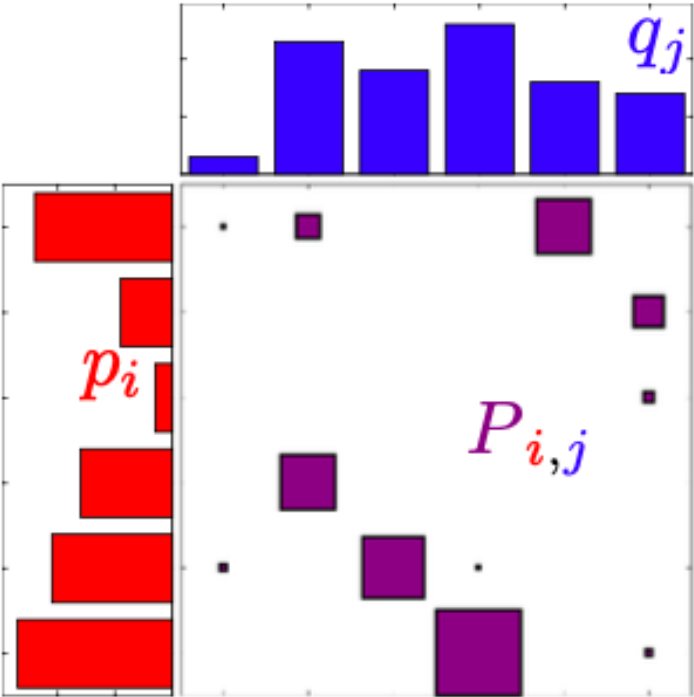
$$\boxed{\mathcal{L}} \sum_j P_{i,j} = p_i$$

$$\boxed{\mathcal{C}} \sum_i P_{i,j} = q_j$$

Deliver the bread to the café, which needs it



	3	23	18	26	16	14
24	1	7	0	0	16	0
9	0	0	0	0	0	9
3	0	0	0	0	0	3
16	0	16	0	0	0	0
21	2	0	18	1	0	0
27	0	0	0	25	0	2



Optimal plan for minimal efforts needed.

Kantorovich: Add Bread Masses

- ▶ $p_i, i \in 1 \dots N$ the mass of bread held by each bakery;
- ▶ $q_j, j \in 1 \dots M$ the mass of bread desired by each cafe;
- ▶ $\sum p_i = \sum q_j$, bread produced = bread needed
- ▶ x_i, y_j the positions of bakeries and cafes;
- ▶ $P_{i,j}$ transport of mass from i to j .
- ▶ $p_i = \sum_j P_{i,j}; q_j = \sum_i P_{i,j}$ limiting capacity exists.
- ▶ Need to solve a problem:

$$\min \sum P_{i,j} c_{i,j}$$

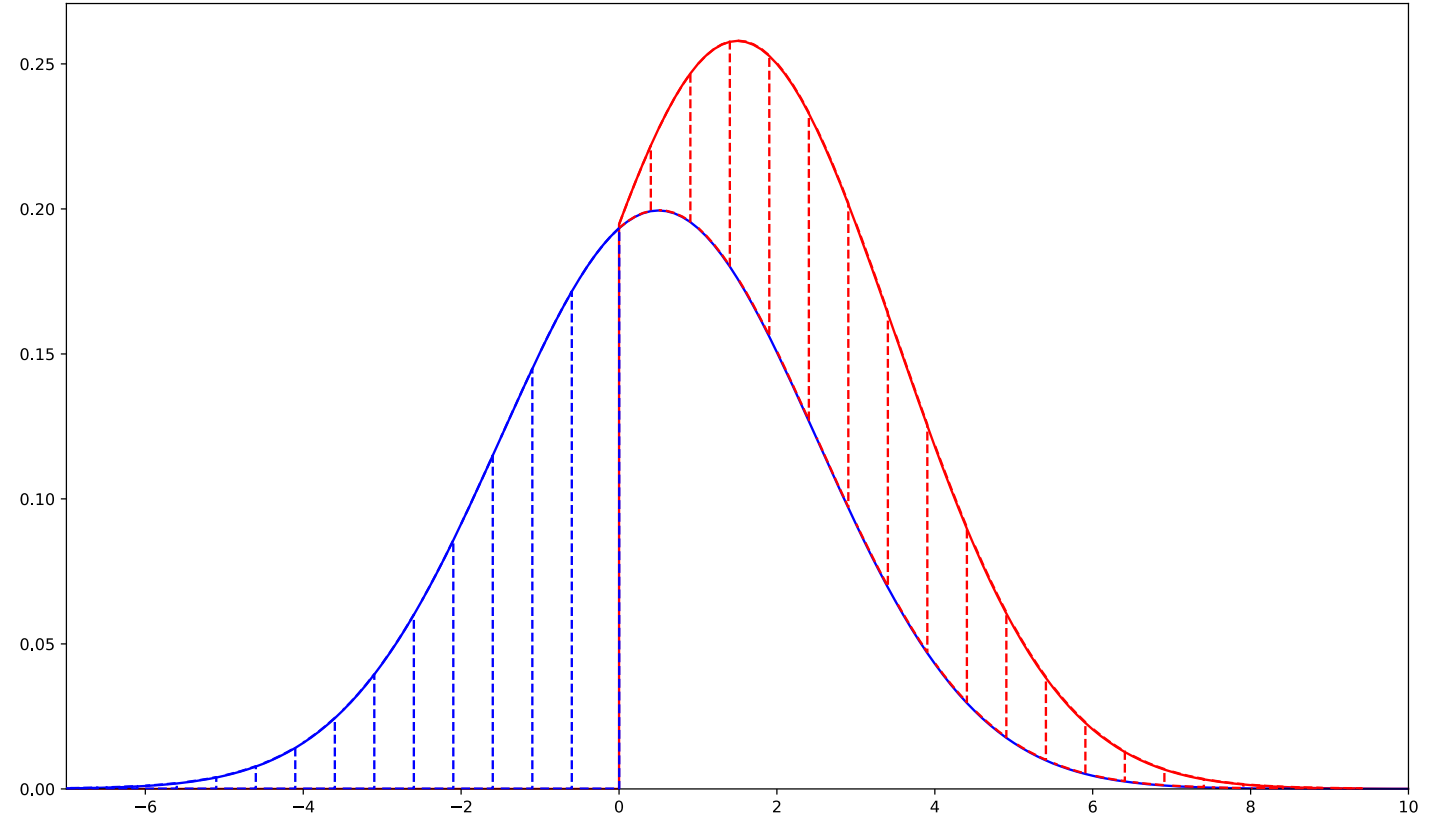
Wasserstein Distance



Wasserstein distance

Also called “Earth mover’s distance” (EMD)

- ▶ Distributions $P(x)$ and $Q(x)$ are viewed as describing the **amounts of “dirt” at point x**
- ▶ We want to convert one distribution into the other by **moving around** some amounts of dirt
- ▶ The cost of moving an amount m from x_1 to x_2 is $m \times \|x_2 - x_1\|$
- ▶ $\text{EMD}(P, Q) = \text{minimum total cost}$ of converting P into Q



Idea of definition

- ▶ Say, we have a moving plan $\gamma(x_1, x_2) \geq 0$:

$\gamma(x_1, x_2)dx_1dx_2$ – how much dirt we're moving from $[x_1, x_1 + dx_1]$ to $[x_2, x_2 + dx_2]$

- ▶ Then, the cost of moving from $[x_1, x_1 + dx_1]$ to $[x_2, x_2 + dx_2]$ is:

$$\|x_2 - x_1\| \cdot \gamma(x_1, x_2)dx_1dx_2$$

- ▶ and the total cost is:

$$C = \int_{x_1, x_2} \|x_2 - x_1\| \cdot \gamma(x_1, x_2)dx_1dx_2 = \mathbb{E}_{x_1, x_2 \sim \gamma(x_1, x_2)} \|x_2 - x_1\|$$

- ▶ Since we want to convert P to Q , the plan has to satisfy:

$$\int_{x_1} \gamma(x_1, x_2)dx_1 = Q(x_2), \quad \int_{x_2} \gamma(x_1, x_2)dx_2 = P(x_1)$$

Idea of Definition

- ▶ Let π be the set of all plans that convert P to Q , i.e.:

$$\pi = \left\{ \gamma: \quad \gamma \geq 0, \quad \int_{x_1} \gamma(x_1, x_2) dx_1 = Q(x_2), \quad \int_{x_2} \gamma(x_1, x_2) dx_2 = P(x_1) \right\}$$

- ▶ Then, the Wasserstein distance between P and Q is:

$$\text{EMD}(P, Q) = \inf_{\gamma \in \pi} \mathbb{E}_{x_1, x_2 \sim \gamma} \|x_2 - x_1\|$$


Optimization over all transport plans – not too friendly

Wasserstein Distance

For continuous case, there are a set of p-Wasserstein distances, with $W_p(p_x, q_y)$ defined with $x \in M, y \in M$ and a distance D on x, y :

$$W_p(p_x, q_y) = \inf_{\gamma \in \Pi(x, y)} \int_{M \times M} D(x, y)^p d\gamma(x, y),$$

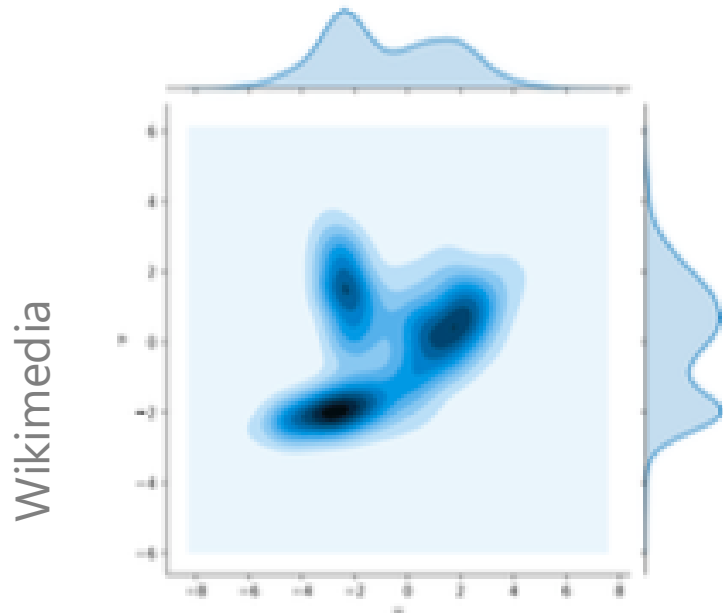
where $\Pi(x, y)$ is a set of all joint distributions having p_x, q_y as their marginals.

W_1 distance

In particular, W_1 distance with Euclidean norm is:

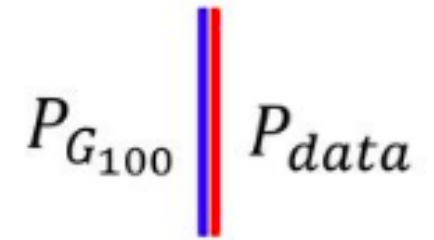
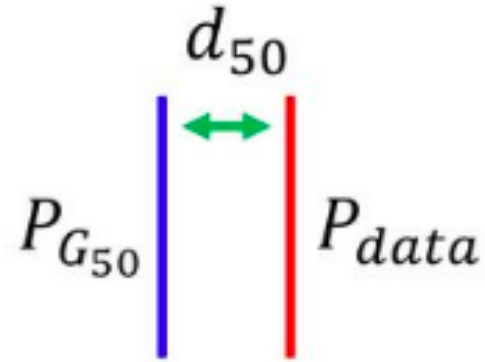
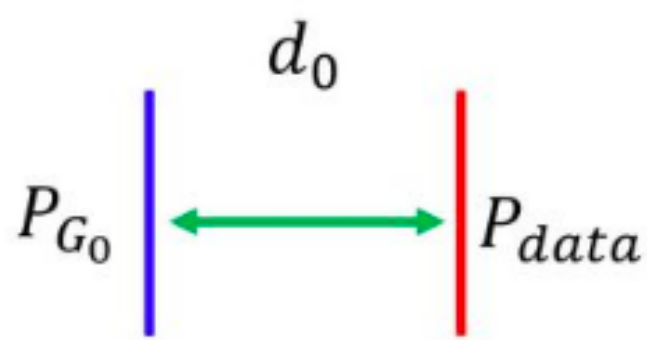
$$W(p_x, q_y) = \inf_{\gamma \in \Pi(x, y)} \int_{M \times M} D(x, y) d\gamma(x, y) = \inf_{\gamma \in \Pi(x, y)} \mathbb{E}(\|x - y\|)$$

Which brings an evident connection to EMD.



Two dimensional representation of the transport plan between horizontal (μ) and vertical ν pdfs. Note, that this is not unique plan. The inf must be taken over all possible plans.

Convergence Example

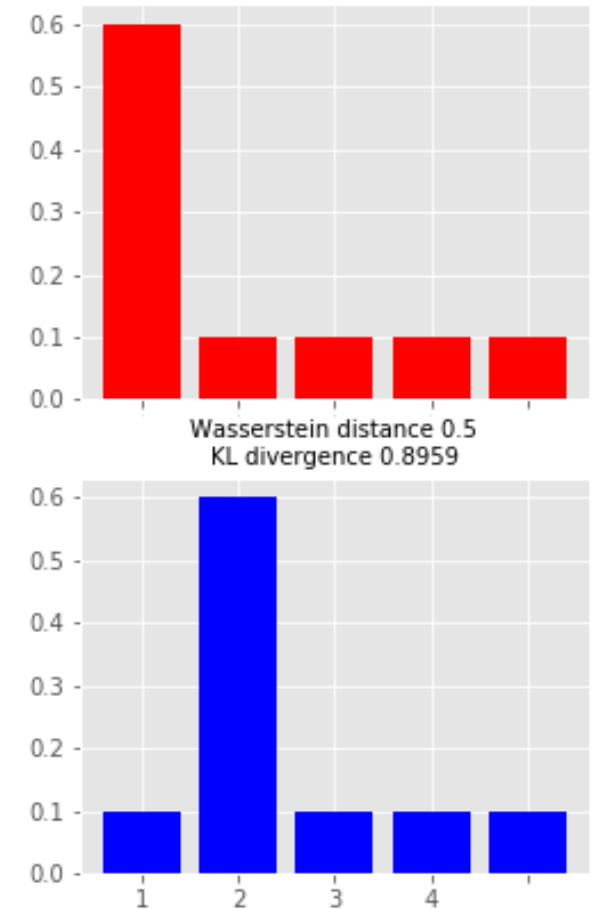
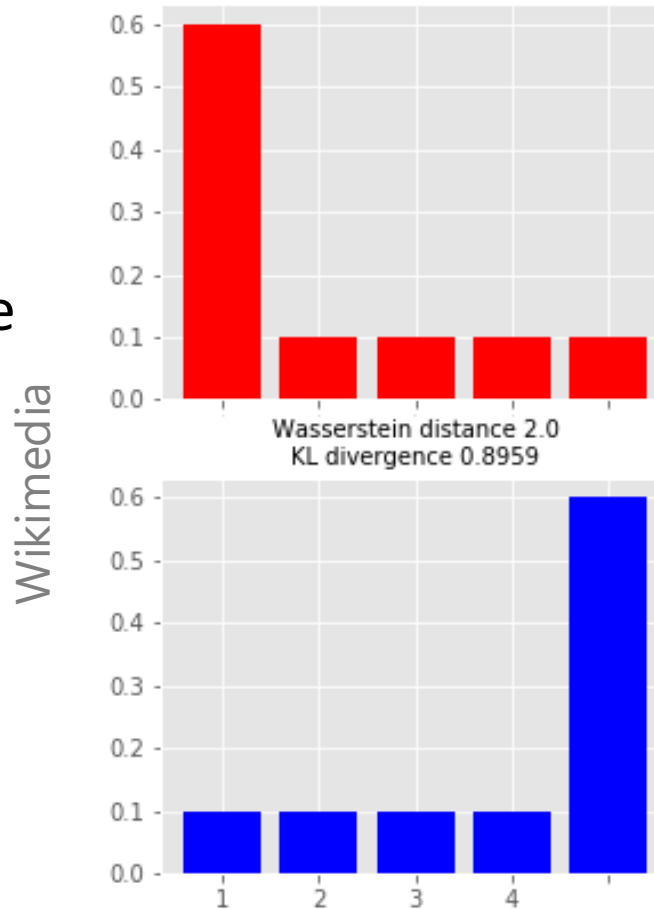


TV:	1	1	0
KL:	∞	∞	0
JS:	$\log 2$	$\log 2$	0
W:	d_0	d_{50}	0

Mass Attention

W takes into account the distance at which the differences in the distributions are located.

This is exactly what we need to take into account multiple solutions!



W properties hints

P – true PDF, Q – fitted PDF.

- ▶ For a sequence of distributions Q_n :

$$KL(P||Q_n) \rightarrow 0 \rightarrow JS(P; Q_n) \rightarrow 0 \rightarrow W(P; Q_n) \rightarrow 0, Q_n \xrightarrow{D} P$$

- ▶ For $Q_\theta \sim g_\theta(z)$, $g_\theta(z)$ continuous

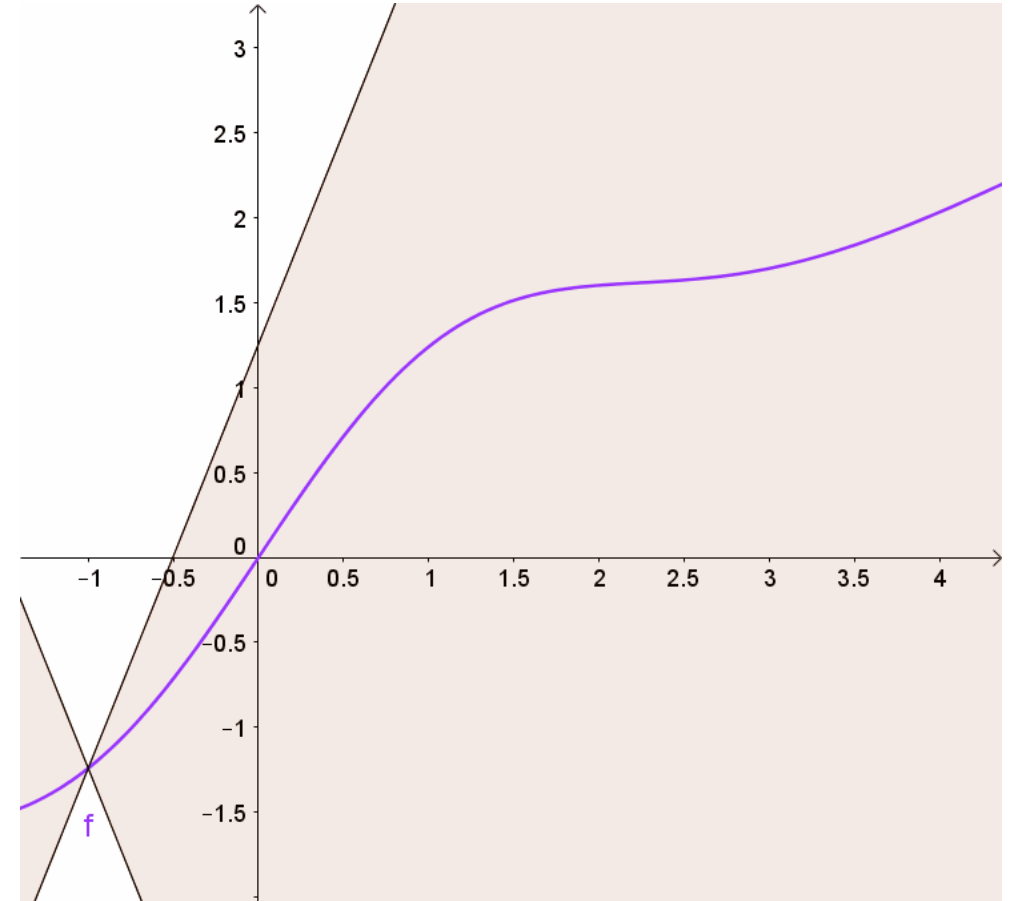
$W(Q_\theta; Q)$ is continuous and can be restricted to differentiable almost everywhere.

Should we use directly in GAN?

Lipschitz continuity

- ▶ f is Lipschitz- k continuous if
- ▶ there exists a constant $k \geq 0$, such that for all x_1 and x_2 :

$$|f(x_1) - f(x_2)| \leq k \cdot \|x_1 - x_2\|$$



img from https://en.wikipedia.org/wiki/Lipschitz_continuity

Kantorovich-Rubinstein Duality

P – true PDF, Q – fitted PDF.

$$W(P; Q) = \sup_{f \in Lip_1} (\mathbb{E}_{x \sim P} f(x) - \mathbb{E}_{x \sim Q} f(x)),$$

where Lip_1 is 1-Lipshitz condition.

Back to Bakeries

In fact, the duality works also for EMD, we can say:

$$EMD = \sup_{||f||_L \leq 1} \sum_i f_i q_i - \sum_j f_j p_j$$

For example of bakeries, f can be interpreted as a price of buying or selling at points x_i and y_j .

Integral Probability Metrics

$p(x), q(x)$ – PDF.

$$\gamma_{\mathcal{F}}(P, Q) = \sup \left\{ \left| \int f dp(x) - \int f dq(x) \right| : f \in \mathcal{F} \right\}$$

$\mathcal{F}$ is a class of real-valued bounded measurable functions on S .

For $\mathcal{F} = \{f : \|f\|_L \leq 1\}$, with 1-Lipschitz condition:

W_1 is **IPM** but **not f -divergence**

B. Sriperumbudur et al. On the empirical estimation of integral probability metrics
S. Nowozin, NIPS2016 workshop talk

Conclusions

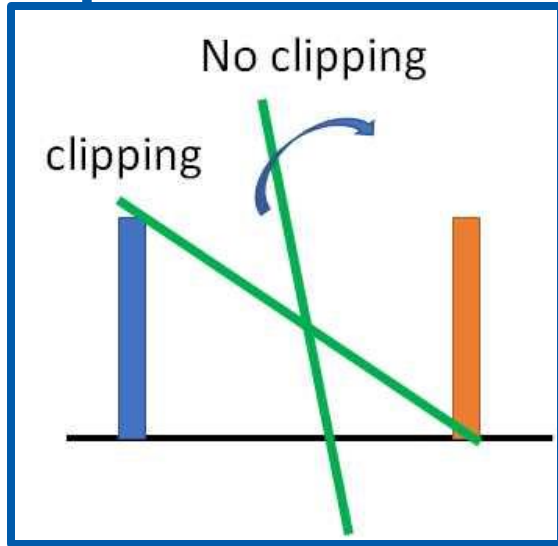
- ▶ Wasserstein-1 is a distance with desired properties.
- ▶ Kantorovich-Rubinstein duality connects Wasserstein-1 distance to IPM.
- ▶ Lipschitzness is needed for above to work.
- ▶ Wasserstein-1 distance cannot directly be inserted into f -GAN style*.

*J. Song et al., Bridging the Gap Between f -GANs and Wasserstein GANs, ICML 2020

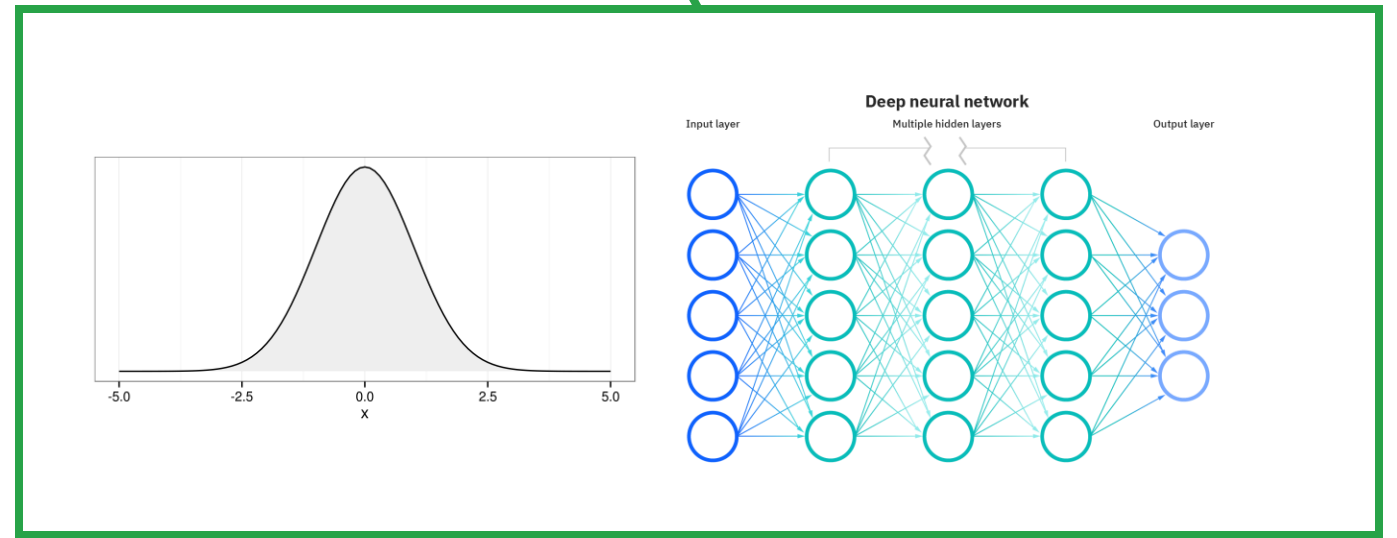
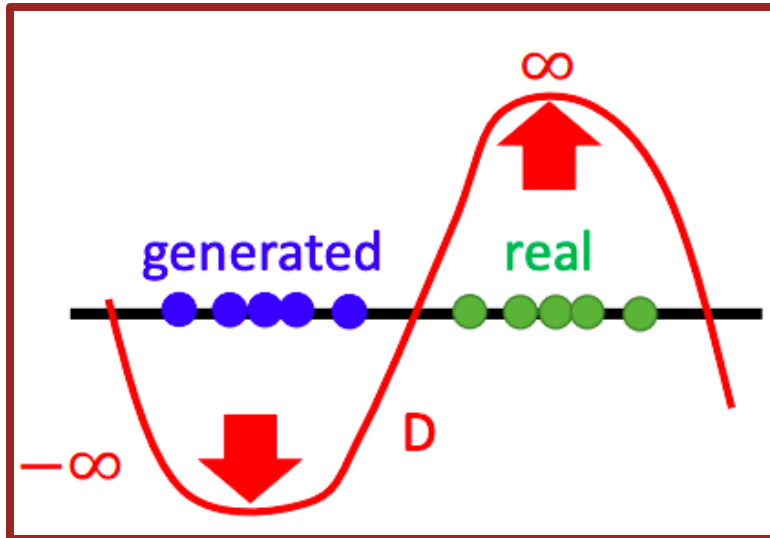
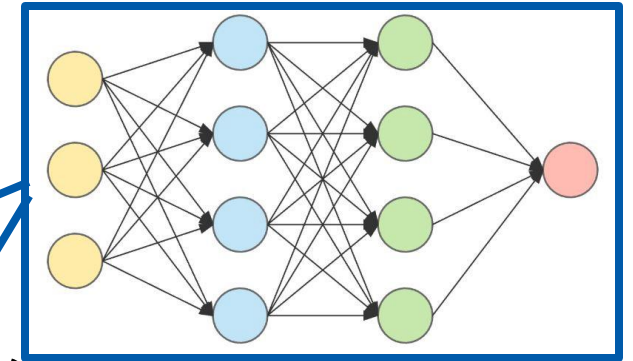
Wasserstein GAN



Lipschitz-1 Condition and Neural Networks



$$W(P; Q) = \sup_{f \in Lip_1} (\mathbb{E}_{x \sim P} f(x) - \mathbb{E}_{x \sim Q} f(x)),$$



Lipschitz-1 Condition and Neural Networks

$$W(P; Q) = \sup_{f \in Lip_1} (\mathbb{E}_{x \sim P} f(x) - \mathbb{E}_{x \sim Q} f(x)),$$

- ▶ f is a neural net – **discriminator** ('**critic**' in the original paper).
- ▶ The expectations is estimated from samples.
- ▶ Lipschitz-1 continuity can be replaced with Lipschitz-k continuity
 - estimate $k \times W(P, Q)$
 - achieved **by clipping the weights** of the critic: $w \rightarrow \text{clip}(w, -c, c)$ with some constant c .

WGAN

Algorithm 1 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

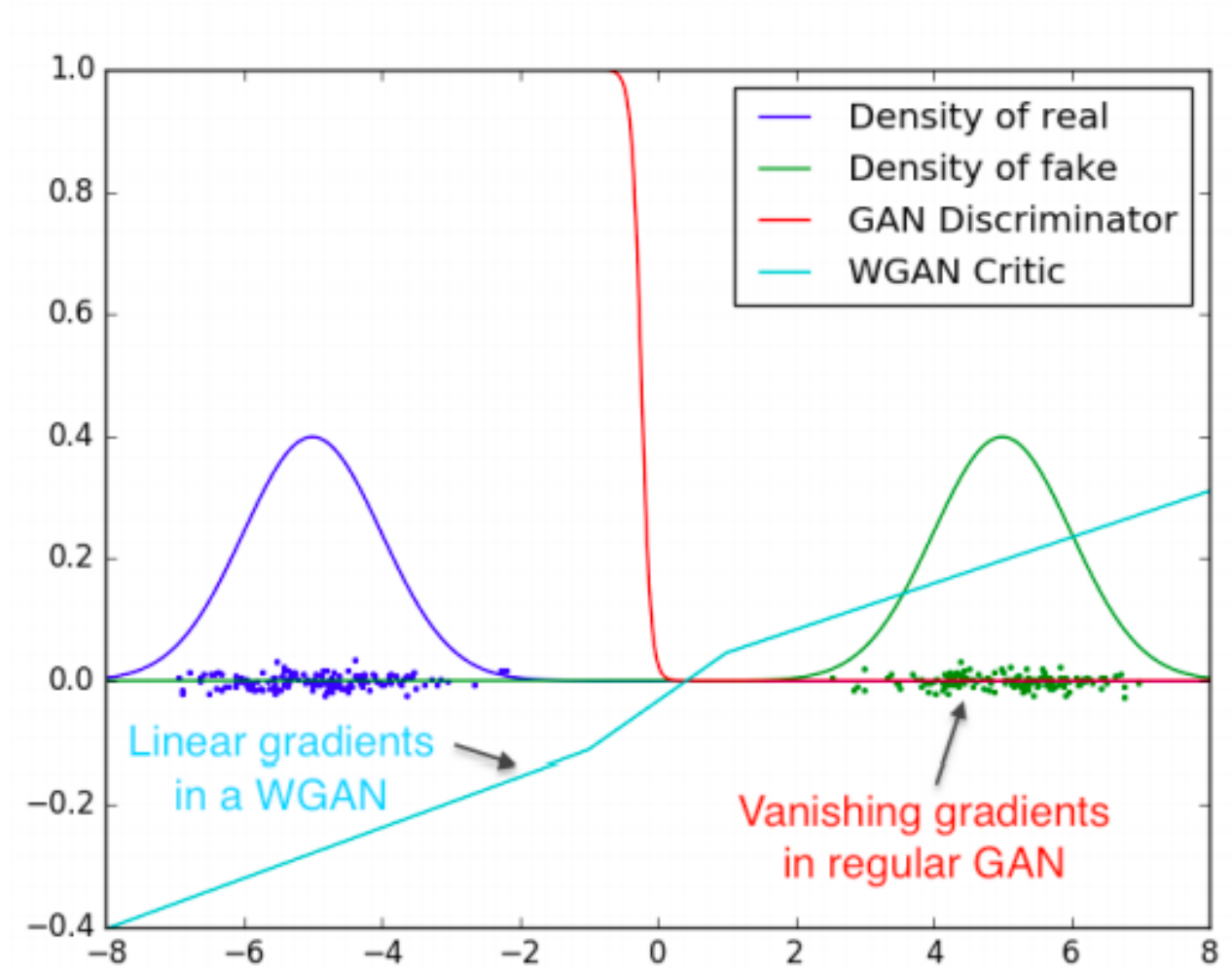
Require: : α , the learning rate. c , the clipping parameter. m , the batch size.
 n_{critic} , the number of iterations of the critic per generator iteration.

Require: : w_0 , initial critic parameters. θ_0 , initial generator's parameters.

```
1: while  $\theta$  has not converged do
2:   for  $t = 0, \dots, n_{\text{critic}}$  do
3:     Sample  $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$  a batch from the real data.
4:     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
5:      $g_w \leftarrow \nabla_w \left[ \frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)})) \right]$ 
6:      $w \leftarrow w + \alpha \cdot \text{RMSPProp}(w, g_w)$ 
7:      $w \leftarrow \text{clip}(w, -c, c)$ 
8:   end for
9:   Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
10:   $g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$ 
11:   $\theta \leftarrow \theta - \alpha \cdot \text{RMSPProp}(\theta, g_\theta)$ 
12: end while
```

WGAN: problems solved

- ▶ the vanishing gradient problem is **solved**;
- ▶ mode collapse problem is **addressed**;
- ▶ From authors: Weight clipping is a clearly terrible way to enforce a Lipschitz constraint. :



WGAN: results



Figure 7: Algorithms trained with an MLP generator with 4 layers and 512 units with ReLU nonlinearities. The number of parameters is similar to that of a DCGAN, but it lacks a strong inductive bias for image generation. Left: WGAN algorithm. Right: standard GAN formulation. The WGAN method still was able to produce samples, lower quality than the DCGAN, and of higher quality than the MLP of the standard GAN. Note the significant degree of mode collapse in the GAN MLP.

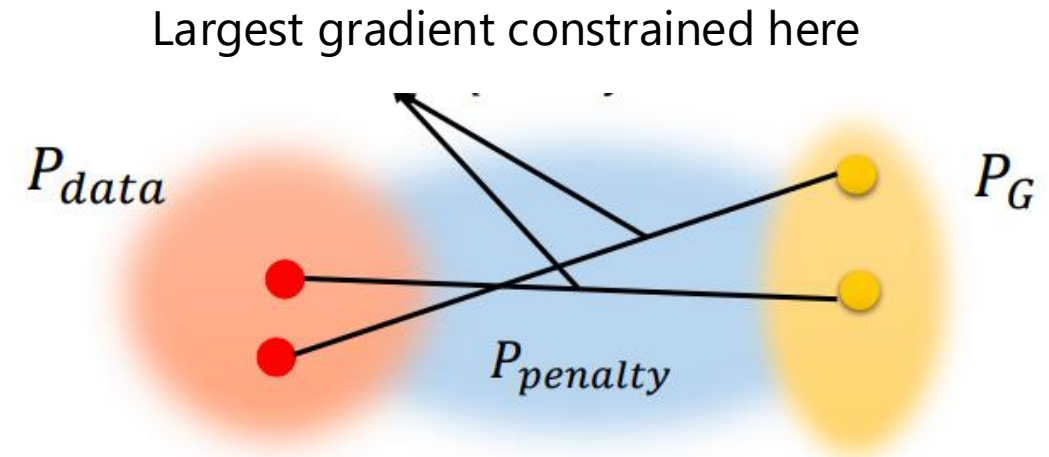
WGAN-GP

- ▶ Weight clipping makes the critic less expressive and the training harder to converge
- ▶ Optimal f should satisfy $\|\nabla f\| = 1$ almost everywhere under P and Q
- ▶ Also: $\|f\|_L \leq 1 \iff \|\nabla f\| \leq 1$
- ▶ Can replace weight clipping with a gradient penalty term:

$$GP = \lambda \int \max[(\|\nabla_{\tilde{x}} f(\tilde{x})\| - 1)^2] dx$$



$$GP = \lambda \mathbb{E}_{\tilde{x} \sim \mathbb{P}_{\tilde{x}}} [(\|\nabla_{\tilde{x}} f(\tilde{x})\| - 1)^2]$$



$$\mathbb{P}_{\tilde{x}} : \begin{bmatrix} \tilde{x} = \alpha x_1 + (1 - \alpha)x_2 \\ \alpha \sim \text{Uniform}(0, 1) \\ x_1 \sim P \\ x_2 \sim Q \end{bmatrix}$$

WGAN-GP

- ▶ Weight clipping makes the critic less expressive and the training harder to converge
- ▶ Optimal f should satisfy $\|\nabla f\| = 1$ almost everywhere under P and Q
- ▶ Also: $\|f\|_L \leq 1 \iff \|\nabla f\| \leq 1$

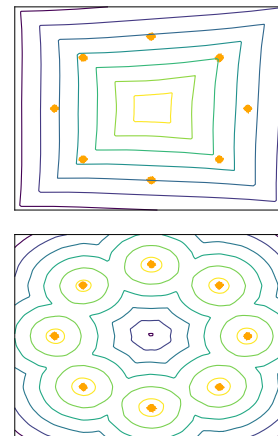
- ▶ Can replace weight clipping with a gradient penalty term:

$$\text{GP} = \lambda \mathbb{E}_{\tilde{x} \sim \mathbb{P}_{\tilde{x}}} [(\|\nabla_{\tilde{x}} f(\tilde{x})\| - 1)^2]$$

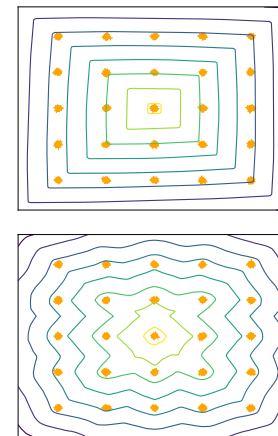
or alternatively ('one-sided' penalty):

$$\text{GP} = \lambda \mathbb{E}_{\tilde{x} \sim \mathbb{P}_{\tilde{x}}} [\max(0, \|\nabla_{\tilde{x}} f(\tilde{x})\| - 1)^2]$$

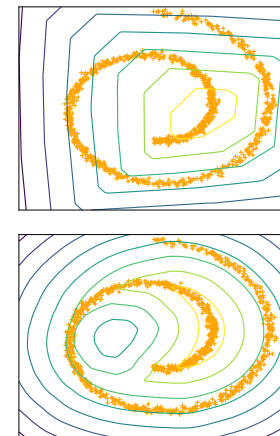
8 Gaussians



25 Gaussians



Swiss Roll



$$\mathbb{P}_{\tilde{x}} : \begin{bmatrix} \tilde{x} = \alpha x_1 + (1 - \alpha)x_2 \\ \alpha \sim \text{Uniform}(0, 1) \\ x_1 \sim P \\ x_2 \sim Q \end{bmatrix}$$

WGAN: spectral normalization

- › Spectral normalisation proposes to use normalised weights:

$$W_{SN} = \frac{W}{\sigma(W)}$$

where:

$$\sigma(W) = \max_{h:h \neq 0} \frac{\|Wh\|_2}{\|h\|_2}$$

- › this gives constraints on gradient:

$$\|f\|_{Lip} \leq \prod_{i=1}^l \sigma(W_l).$$

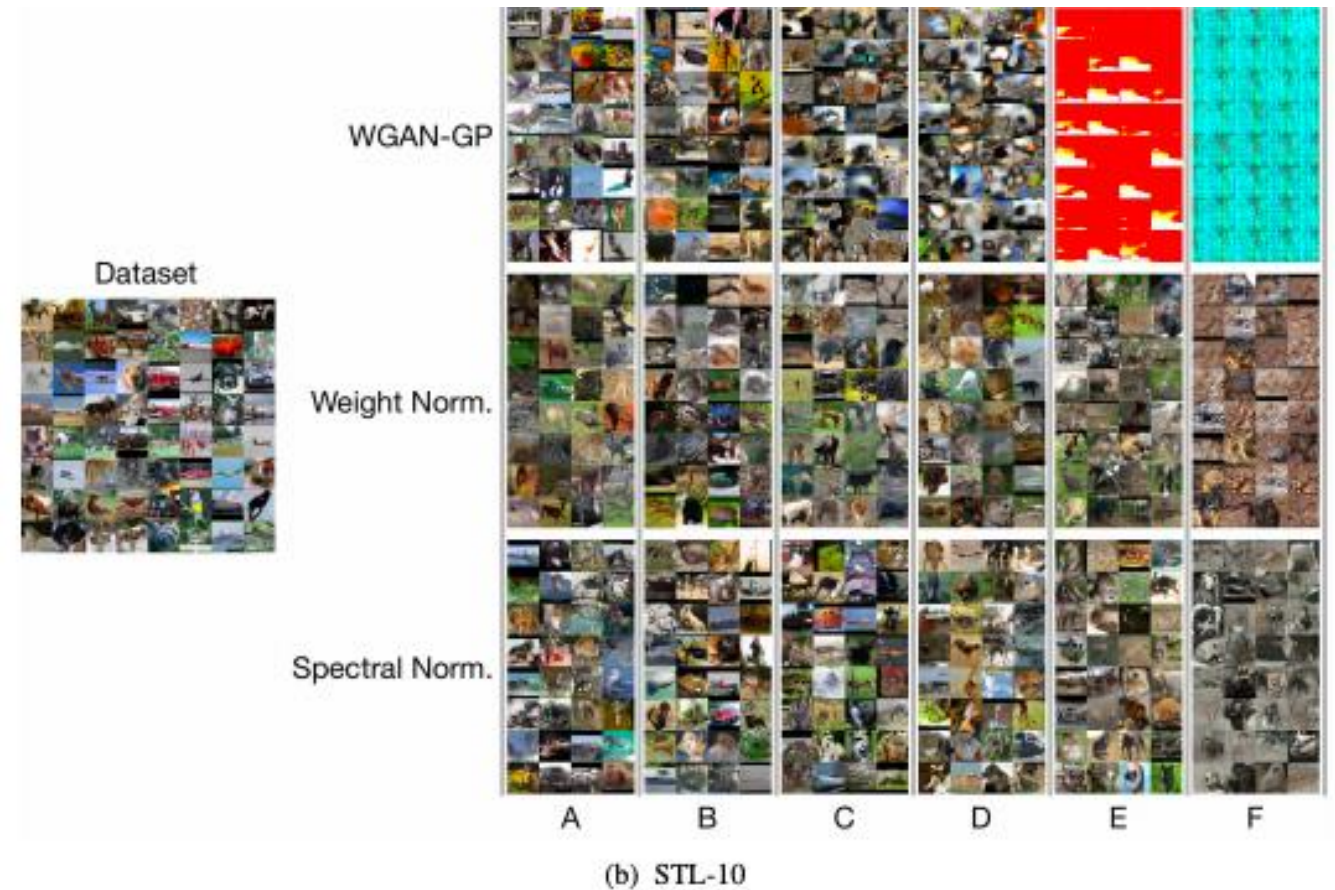
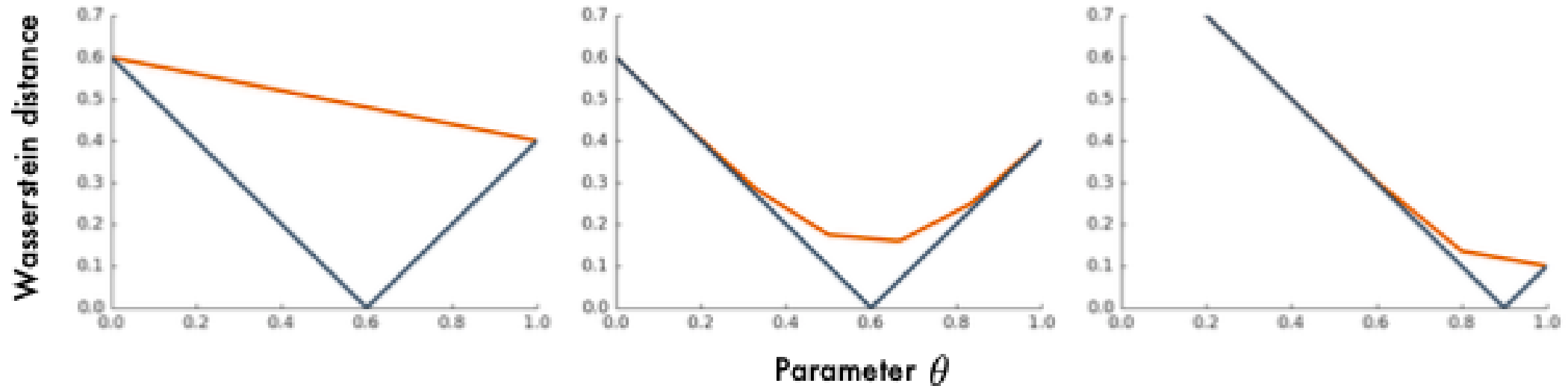


Figure 6: Generated images on different methods: WGAN-GP, weight normalization, and spectral normalization on CIFAR-10 and STL-10.

Miyato et al. Spectral Normalization for Generative Adversarial Networks, ICLR 2018

WGAN: problems

- › The expected EMD gradients can differ from the true gradients.
- › This leads to problems even for Bernoulli distribution.



Red for sample gradient expectation, blue is for real gradients solution.
Left to right $\theta^* = 0.6; 0.6; 0.9$.

M. Bellemare et al. The Cramer Distance as a Solution to Biased Wasserstein Gradients

Conclusions

- ▶ WGAN is a powerful generative model.
- ▶ Simpler training procedure but need to control Lipschitz continuity
- ▶ Several ideas how to do this.
- ▶ Still problems:
 - Kantorovich-Rubinstein duality only mimicked;
 - gradient is stuck near solutions.